

Chapter 6: Data Manipulation



Outlines

- Introduction
- Basic data structures in pandas
- Essential basic functionality
- Data Indexing and Selection
- Hierarchical Indexing
- Operating on Data in Pandas
- Handling Missing Data
- Combining Datasets
- Aggregation and Grouping
- `pivot()` and `pivot_table()`

Introduction

- Pandas is a Python library used for working with data sets.
- It has functions for analyzing, cleaning, exploring, and manipulating data.

```
import numpy as np  
import pandas as pd
```

Basic data structures in pandas

Pandas provides two types of classes for handling data:

- Series: a one-dimensional labeled array holding data of any type such as integers, strings, Python objects etc.
- DataFrame: a two-dimensional data structure that holds data like a two-dimensional array or a table with rows and columns.

Basic data structures in pandas

- Creating a Series by passing a list of values, letting pandas create a default RangeIndex.

```
s = pd.Series([1, 3, 5, np.nan, 6])
```

- Creating a DataFrame by passing a dictionary of objects where the keys are the column labels and the values are the column values.

```
df = pd.DataFrame(  
    {"AAA": [4, 5, 6, 7, 8, 9],  
     "BBB": [10, 20, 30, 40, 50, 60],  
     "CCC": [100, 50, -30, -50, -40, -80]}  
)
```

	AAA	BBB	CCC
1	5	20	50
2	6	30	-30
3	7	40	-50
4	8	50	-40
5	9	60	-80

Essential basic functionality

Essential basic functionality

Head and tail: to view a small sample of a Series or DataFrame object

```
s.head() # 5 first sample in default  
df.tail(3)
```

Attributes and underlying data: to access the metadata

shape: gives the axis dimensions of the object, consistent with ndarray

Axis labels

Series: **index** (only axis)

DataFrame: **index** (rows) and **columns**

```
s.shape # (5,)
```

```
print(df.shape) # (6, 3)
```

```
s.index # RangeIndex(start=0, stop=5, step=1)
```

```
df.index # RangeIndex(start=0, stop=6, step=1)
```

```
df.columns # Index(['AAA', 'BBB', 'CCC'], dtype='object')
```

Essential basic functionality

Return a NumPy representation of the underlying data

`df.to_numpy()`

```
[[ 4 10 100]
 [ 5 20 50]
 [ 6 30 -30]
 [ 7 40 -50]
 [ 8 50 -40]
 [ 9 60 -80]]
```

`describe()` shows a quick statistic summary of your data

`df.describe()`

	AAA	BBB	CCC
count	6.000000	6.000000	6.000000
mean	6.500000	35.000000	-8.333333
std	1.870829	18.708287	68.532231
min	4.000000	10.000000	-80.000000
25%	5.250000	22.500000	-47.500000
50%	6.500000	35.000000	-35.000000
75%	7.750000	47.500000	30.000000
max	9.000000	60.000000	100.000000

Essential basic functionality

`DataFrame.sort_index()` sorts by an axis

`df.sort_index(axis=1, ascending=False)`

	CCC	BBB	AAA
0	100	10	4
1	50	20	5
2	-30	30	6
3	-50	40	7
4	-40	50	8
5	-80	60	9

`DataFrame.sort_values()` sorts by values

`df.sort_values(by="BBB")`

	AAA	BBB	CCC
0	4	10	100
1	5	20	50
2	6	30	-30
3	7	40	-50
4	8	50	-40
5	9	60	-80

Data Indexing and Selection

Data Selection in Series

Data Selection in Dataframes

Data Selection in Series

Series as dictionary

```
import pandas as pd
data = pd.Series([0.25, 0.5, 0.75, 1.0],
                 index=['a', 'b', 'c', 'd'])
print(data)
print(data['b'])
print(data.keys())
print(list(data.items()))
```

```
# data
a 0.25
b 0.50
c 0.75
d 1.00
dtype: float64

# data['b']
0.5

# data.keys()
Index(['a', 'b', 'c', 'd'], dtype='object')

# list(data.items())
[('a', 0.25), ('b', 0.5), ('c', 0.75), ('d', 1.0)]
```

Data Selection in Series

Series as one-dimensional array

slicing by explicit index

```
print(data['a':'c'])
```

slicing by implicit integer index

```
print(data[0:2])
```

masking

```
print(data[(data > 0.3) & (data < 0.8)])
```

fancy indexing

```
print(data[['a', 'e']])
```

```
# data['a':'c']  
a 0.25  
b 0.50  
c 0.75  
dtype: float64
```

```
# data[0:2]  
a 0.25  
b 0.50  
dtype: float64
```

```
# data[(data > 0.3) & (data < 0.8)]  
b 0.50  
c 0.75  
dtype: float64
```

```
# data[['a', 'e']]  
a 0.25  
e 1.25  
dtype: float64
```

```
# data  
a 0.25  
b 0.50  
c 0.75  
d 1.00  
dtype: float64
```

Data Selection in Series

Indexers: loc, iloc, and ix

```
data = pd.Series(['a', 'b', 'c', 'd'],  
                 index=[2, 4, 6, 8])
```

#references the explicit index

```
print(data.loc[2])  
print(data.loc[2:6])
```

#references the implicit Python-style index,
starting from 0

```
print(data.iloc[1])  
print(data.iloc[1:3])
```

```
print(data.loc[1])  
'a'
```

```
print(data.loc[1:3])  
1      a  
3      b  
dtype: object
```

```
print(data.iloc[1])  
'b'
```

```
print(data.iloc[1:3])  
3      b  
5      c  
dtype: object
```

Data Selection in DataFrame

DataFrame as a dictionary

```
area = pd.Series({'California': 423967, 'Texas':  
695662, 'New York': 141297, 'Florida': 170312,  
'Illinois': 149995})
```

```
pop = pd.Series({'California': 38332521, 'Texas':  
26448193, 'New York': 19651127, 'Florida':  
19552860, 'Illinois': 12882135})
```

```
data = pd.DataFrame({'area':area, 'pop':pop})
```

```
data['area']
```

```
data.area
```

```
data['density'] = data['pop'] / data['area']
```

```
# data['area']
```

```
California    423967
```

```
Florida       170312
```

```
Illinois      149995
```

```
New York     141297
```

```
Texas        695662
```

```
Name: area, dtype: int64
```

```
# data
```

	area	pop	density
California	423967	38332521	90.413926
Texas	695662	26448193	38.018740
New York	141297	19651127	139.076746
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763

Data Selection in DataFrame

DataFrame as two-dimensional array

data

	area	pop	density
California	423967	38332521	90.413926
Texas	695662	26448193	38.018740
New York	141297	19651127	139.076746
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763

data.values

```
[[4.23967000e+05 3.83325210e+07 9.04139261e+01]
 [6.95662000e+05 2.64481930e+07 3.80187404e+01]
 [1.41297000e+05 1.96511270e+07 1.39076746e+02]
 [1.70312000e+05 1.95528600e+07 1.14806121e+02]
 [1.49995000e+05 1.28821350e+07 8.58837628e+01]]
```

data.T

	California	Texas	New York	Florida	Illinois
area	4.239670e+05	6.956620e+05	1.412970e+05	1.703120e+05	1.499950e+05
pop	3.833252e+07	2.644819e+07	1.965113e+07	1.955286e+07	1.288214e+07
density	9.041393e+01	3.801874e+01	1.390767e+02	1.148061e+02	8.588376e+01

Data Selection in DataFrame

DataFrame as two-dimensional array

	area	pop	density
California	423967	38332521	90.413926
Texas	695662	26448193	38.018740
New York	141297	19651127	139.076746
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763

data

data.iloc[:3, :2]

data.loc[:'New York', :'pop']

	area	pop
California	423967	38332521
Texas	695662	26448193
New York	141297	19651127

Data Selection in DataFrame

Data Selection in DataFrame

Additional indexing conventions

`data`

	area	pop	density
California	423967	38332521	90.413926
Texas	695662	26448193	38.018740
New York	141297	19651127	139.076746
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763

`data['Texas':'Florida']`

`data[1:4]`

	area	pop	density
Texas	695662	26448193	38.018740
New York	141297	19651127	139.076746
Florida	170312	19552860	114.806121

`data[data.density > 100]`

	area	pop	density
New York	141297	19651127	139.076746
Florida	170312	19552860	114.806121

Hierarchical Indexing

MultiIndex object typically stores the axis labels in pandas objects

MultiIndex Creation

A MultiIndex can be created from a list of arrays, an array of tuples, a crossed set of iterables, or a DataFrame

```
arrays = [ ["bar", "bar", "foo", "foo"],  
           ["one", "two", "one", "two"], ]  
tuples = list(zip(*arrays))  
index = pd.MultiIndex.from_tuples(tuples,  
                                  names=["first", "second"])
```

```
iterables = [ ["bar", "foo"], ["one", "two"] ]  
index = pd.MultiIndex.from_product(iterables,  
                                   names=["first", "second"])
```

```
df = pd.DataFrame([ ["bar", "one"], ["bar", "two"],  
                    ["foo", "one"], ["foo", "two"] ],  
                  columns=["first", "second"])  
index = pd.MultiIndex.from_frame(df)
```

```
s = pd.Series(np.random.randn(4), index=index)
```

	first	second	
	bar	one	0.555879
		two	1.220156
	foo	one	0.585259
		two	0.808811
			dtype: float64

MultiIndex Creation

Simply pass a list of two or more index arrays to the constructor.

```
arrays = [["bar", "bar", "foo", "foo"],  
          ["one", "two", "one", "two"],]  
s = pd.Series(np.random.randn(4), index=arrays)  
df = pd.DataFrame(np.random.randn(4, 2), index=arrays)
```

```
bar one    1.143852
```

```
      two -0.047293
```

```
foo one  -0.250895
```

```
      two -1.273642
```

```
dtype: float64
```

		0	1
bar	one	0.205028	2.215562
	two	0.257963	-0.053902
foo	one	1.929341	-0.481944
	two	-1.784082	-0.995362

MultiIndex Creation

- Pass a dictionary with appropriate tuples as keys, Pandas automatically recognize this and use a MultiIndex by default.

```
data = {('California', 2000): 33871648,  
        ('California', 2010): 37253956,  
        ('Texas', 2000): 20851820,  
        ('Texas', 2010): 25145561,  
        ('New York', 2000): 18976457,  
        ('New York', 2010): 19378102}
```

```
pop = pd.Series(pop)
```

- MultiIndex level names

```
pop.index.names = ['state', 'year']
```

California	2000	33871648
	2010	37253956
Texas	2000	20851820
	2010	25145561
New York	2000	18976457
	2010	19378102

dtype: int64

Basic indexing

Select data by a “partial” label identifying a subgroup in the data

```
df = pd.DataFrame(np.random.randn(4, 3), index=index)
print(df)
print(df.loc["bar"])
```

```
#df
```

		0	1	2
first	second			
bar	one	-0.839371	-0.339005	-0.220530
	two	0.836290	-0.837688	1.231631
foo	one	-0.133312	0.388936	0.417612
	two	-2.010114	-0.399452	-1.548661

```
# df.loc["bar"]
```

	0	1	2
second			
one	-0.839371	-0.339005	-0.220530
two	0.836290	-0.837688	1.231631

Advanced indexing

```
df = pd.DataFrame(np.random.randn(4, 3),  
                  index=index)
```

```
print(df)
```

```
print(df.loc["bar", "one"])
```

```
print(df.loc["bar":"foo"])
```

	0	1	2	
first	second			
bar	one	-2.344333	0.568462	1.464509
	two	-1.388778	-0.882005	-1.572366
foo	one	-0.269053	-1.288837	1.399942
	two	-0.077181	-0.201282	-1.508854

```
# df.loc["bar", "one"]
```

```
0 -2.344333
```

```
1 0.568462
```

```
2 1.464509
```

```
Name: (bar, one), dtype: float64
```

```
#df.loc["bar":"foo"]
```

		0	1	2
first	second			
bar	one	-2.344333	0.568462	1.464509
	two	-1.388778	-0.882005	-1.572366
foo	one	-0.269053	-1.288837	1.399942
	two	-0.077181	-0.201282	-1.508854

Advanced indexing

```
df = pd.DataFrame(np.random.randn(4, 3),  
                  index=index)
```

```
print(df)
```

```
print(df.loc[("bar", "one"):(("foo", "one"))])
```

```
print(df.loc[("bar", "one"):"foo"])
```

	0	1	2
first second			
bar one	-2.344333	0.568462	1.464509
two	-1.388778	-0.882005	-1.572366
foo one	-0.269053	-1.288837	1.399942
two	-0.077181	-0.201282	-1.508854

	0	1	2
first second			
bar one	-2.344333	0.568462	1.464509
two	-1.388778	-0.882005	-1.572366
foo one	-0.269053	-1.288837	1.399942
	0	1	2
first second			
bar one	-2.344333	0.568462	1.464509
two	-1.388778	-0.882005	-1.572366
foo one	-0.269053	-1.288837	1.399942
two	-0.077181	-0.201282	-1.508854

Operating on Data in Pandas

Ufuncs: Index Preservation

UFuncs: Index Alignment

Ufuncs: Operations Between DataFrame and Series

Ufuncs: Index Preservation

If we apply a NumPy ufunc on either of these objects, the result will be another Pandas object with the indices preserved:

```
df = pd.DataFrame([[6,3,7,4], [6,9,2,6],[7,4,3,7]], columns=['A', 'B', 'C', 'D'])  
print(df)  
print(np.sum(df/ 4))
```

	A	B	C	D
0	6	3	7	4
1	6	9	2	6
2	7	4	3	7

A	4.75
B	4.00
C	3.00
D	4.25

dtype: float64

UFuncs: Index Alignment

For binary operations on two Series or DataFrame objects, Pandas will align indices in the process of performing the operation.

Index alignment in Series

Any item for which one or the other does not have an entry is marked with NaN

```
area = pd.Series({'Alaska': 1723337, 'Texas': 695662, 'California': 423967}, name='area')
population = pd.Series({'California': 38332521, 'Texas': 26448193, 'New York': 19651127},
name='population')
print(population / area)
```

```
Alaska      NaN
California   90.413926
New York     NaN
Texas       38.018740
dtype: float64
```

UFuncs: Index Alignment

Index alignment in Series

Any item for which one or the other does not have an entry is marked with NaN

```
A = pd.Series([2, 4, 6], index=[0, 1, 2])
```

```
B = pd.Series([1, 3, 5], index=[1, 2, 3])
```

```
print(A + B)
```

```
0    NaN
```

```
1     5.0
```

```
2     9.0
```

```
3    NaN
```

```
dtype: float64
```

UFuncs: Index Alignment

Index alignment in DataFrame

A similar type of alignment takes place for both columns and indices when you are performing operations on DataFrames

```
df1 = pd.DataFrame([[6, 3, 7], [6, 9, 2]], columns=['A', 'B', 'C'])  
df2 = pd.DataFrame([[1, 2], [3, 4], [5, 6]], columns=['A', 'B'])  
print(df1 + df2)
```

	A	B	C
0	7.0	5.0	NaN
1	9.0	13.0	NaN
2	NaN	NaN	NaN

UFuncs: Index Alignment

```
df1 = pd.DataFrame([[6, 3, 7], [6, 9, 2]], columns=['A', 'B', 'C'])
df2 = pd.DataFrame([[1, 2], [3, 4], [5, 6]], columns=['A', 'B'])
print(df1 + df2)
df1.add(df2, fill_value=0)
```

Table 3-1. Mapping between Python operators and Pandas methods

Python operator	Pandas method(s)
+	add()
-	sub(), subtract()
*	mul(), multiply()
/	truediv(), div(), divide()
//	floordiv()
%	mod()
**	pow()

Ufuncs: Operations Between DataFrame and Series

When you are performing operations between a DataFrame and a Series, the index and column alignment is similarly maintained

```
df1 = pd.DataFrame([[6, 3, 7], [6, 9, 2]], index=["00", "01"],
                    columns=['A', 'B', 'C'])

print(df1 - df1.iloc[0])

print(df1.subtract(df1.loc['00']))
```

Handling Missing Data

Missing Data in Pandas

Operating on Null Values

Missing Data in Pandas

None: Pythonic missing data

Arrays of Python objects

```
vals1 = np.array([1, None, 3, 4])  
print(vals1, vals1.dtype)  
# [1 None 3 4] object
```

- perform aggregations like `sum()`, `min()`, ...: slowly
- perform aggregations across an array with a `None` value: error

Missing Data in Pandas

NaN: Missing numerical data

```
vals2 = np.array([1, np.nan, 3, 4])  
print(vals2, vals2.dtype)  
# [ 1. nan  3.  4.] float64
```

NaN is specifically a floating-point value

The result of arithmetic with NaN will be NaN

```
print(1 + np.nan) # nan  
print(vals2.sum(), vals2.min(), vals2.max()) # nan nan nan
```

Some special aggregations that will ignore these missing values

```
print(vals2.sum(), vals2.min(), vals2.max()) # 8.0 1.0 4.0
```

Missing Data in Pandas

NaN and None in Pandas

Pandas automatically type-casts when NA values are present

```
ser1 = pd.Series([1, np.nan, 2, None])  
print(ser1)
```

```
0    1.0  
1    NaN  
2    2.0  
3    NaN  
dtype: float64
```

Table 3-2. Pandas handling of NAs by type

Typeclass	Conversion when storing NAs	NA sentinel value
floating	No change	<code>np.nan</code>
object	No change	<code>None</code> or <code>np.nan</code>
integer	Cast to <code>float64</code>	<code>np.nan</code>
boolean	Cast to object	<code>None</code> or <code>np.nan</code>

Operating on Null Values

Detecting null values

detecting null data: `isnull()` and `notnull()`

```
data = pd.Series([1, np.nan, 'hello', None])  
print(data[data.isnull()])  
print(data[data.notnull()])  
print(data.dropna())
```

```
1    NaN  
3    None  
dtype: object
```

```
0    1  
2  hello  
dtype: object
```

```
0    1  
2  hello  
dtype: object
```

Operating on Null Values

Dropping null values

```
df = pd.DataFrame([[1, np.nan, 2], [2,
3, 5], [np.nan, 4, 6]])
print(df.dropna())
print(df.dropna(axis='columns',
how='all'))
print(df.dropna(axis='rows',
thresh=3))
```

```
df.dropna()
```

	0	1	2
1	2.0	3.0	5

```
df.dropna(axis='columns', how='all')
```

	0	1	2
0	1.0	NaN	2
1	2.0	3.0	5
2	NaN	4.0	6

```
df.dropna(axis='rows', thresh=3)
```

	0	1	2
1	2.0	3.0	5

Operating on Null Values

Filling null values

```
data = pd.Series([1, np.nan, 2, None, 3],  
                 index=list('abcde'))  
  
print(data.fillna(0)) # replace na by 0  
  
print(data.ffill())  
  
print(data.bfill())
```

```
a  1.0  
b  1.0  
c  2.0  
d  2.0  
e  3.0  
dtype: float64
```

```
a  1.0  
b  2.0  
c  2.0  
d  3.0  
e  3.0  
dtype: float64
```

Combining Datasets

Combining Datasets

Simple Concatenation with `pd.concat`

```
ser1 = pd.Series(['A', 'B', 'C'], index=[1, 2, 3])
ser2 = pd.Series(['D', 'E', 'F'], index=[4, 5, 6])
print(pd.concat([ser1, ser2]))
```

```
1  A
2  B
3  C
4  D
5  E
6  F
dtype: object
```

Combining Datasets

Simple Concatenation with `pd.concat`

```
df1 = make_df('AB', [1, 2])  
df2 = make_df('AB', [3, 4])  
print(df1)  
print(df2)  
print(pd.concat([df1, df2]))
```

```
# df1  
  A  B  
1 A1 B1  
2 A2 B2
```

```
# df2  
  A  B  
3 A3 B3  
4 A4 B4
```

```
# pd.concat([df1, df2])  
  A  B  
1 A1 B1  
2 A2 B2  
3 A3 B3  
4 A4 B4
```

Combining Datasets

Duplicate indices - pd.concat

```
df1 = make_df('AB', [1, 2])  
df2 = make_df('AB', [1, 2])  
print(df1)  
print(df2)  
print(pd.concat([df1, df2]))
```

```
# df1  
  A  B  
1 A1 B1  
2 A2 B2
```

```
# df2  
  A  B  
1 A1 B1  
2 A2 B2
```

```
# df3  
  A  B  
1 A1 B1  
2 A2 B2  
1 A1 B1  
2 A2 B2
```

Combining Datasets

Ignoring the index - pd.concat

```
df1 = make_df('AB', [1, 2])
df2 = make_df('AB', [1, 2])
print(df1)
print(df2)
print(pd.concat([df1, df2],
                 ignore_index=True))
```

```
# df1
  A  B
1 A1 B1
2 A2 B2
```

```
# df2
  A  B
1 A1 B1
2 A2 B2
```

```
# pd.concat([df1, df2], ignore_index=True)
```

```
  A  B
0 A1 B1
1 A2 B2
2 A1 B1
3 A2 B2
```

Combining Datasets

Concatenation with joins - pd.concat

```
df5 = make_df('ABC', [1, 2])
```

```
df6 = make_df('BCD', [3, 4])
```

```
print(df5)
```

```
print(df6)
```

```
print(pd.concat([df5, df6]))
```

```
print(pd.concat([df5, df6],  
                join='inner'))
```

```
# df5
```

	A	B	C
1	A1	B1	C1
2	A2	B2	C2

```
# df6
```

	B	C	D
3	B3	C3	D3
4	B4	C4	D4

```
# pd.concat([df5, df6])
```

	A	B	C	D
1	A1	B1	C1	NaN
2	A2	B2	C2	NaN
3	NaN	B3	C3	D3
4	NaN	B4	C4	D4

```
# pd.concat([df5, df6], join='inner')
```

	B	C
1	B1	C1
2	B2	C2
3	B3	C3
4	B4	C4

Combining Datasets

```
pd.merge(df1, df2)
```

```
# pd.merge(df1, df2)
```

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

```
# df1
```

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR

```
# df2
```

	employee	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012
3	Sue	2014

Combining Datasets

Specification of the Merge Key

```
print(pd.merge(df1, df2, on='employee'))
```

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

```
# df1
```

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR

```
# df2
```

	employee	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012
3	Sue	2014

Combining Datasets

Merge two datasets with different column names;

```
df4 = pd.merge(df1, df3,  
               left_on="employee",  
               right_on="name")  
df4.drop("name", axis=1)
```

	employee	group	name	salary
0	Bob	Accounting	Bob	70000
1	Jake	Engineering	Jake	80000
2	Lisa	Engineering	Lisa	120000
3	Sue	HR	Sue	90000

```
# df1  
employee  group  
0   Bob  Accounting  
1   Jake Engineering  
2   Lisa Engineering  
3   Sue      HR
```

```
# df2  
name salary  
0  Bob  70000  
1  Jake  80000  
2  Lisa 120000  
3  Sue  90000
```

Combining Datasets

Merge on an index

```
df1a = df1.set_index('employee')
df2a = df2.set_index('employee')
df3a = pd.merge(df1a, df2a,
                 left_index=True, right_index=True)
```

	group	hire_date
employee		
Bob	Accounting	2008
Jake	Engineering	2012
Lisa	Engineering	2004
Sue	HR	2014

```
# df1a
      group
employee
Bob    Accounting
Jake   Engineering
Lisa   Engineering
Sue        HR
```

```
#df2a
      hire_date
employee
Lisa    2004
Bob     2008
Jake    2012
Sue     2014
```

Combining Datasets

join(): merge on indices

```
df3b=df1a.join(df2a)
```

	group	hire_date
employee		
Bob	Accounting	2008
Jake	Engineering	2012
Lisa	Engineering	2004
Sue	HR	2014

```
# df1a
```

	group
employee	
Bob	Accounting
Jake	Engineering
Lisa	Engineering
Sue	HR

```
#df2a
```

	hire_date
employee	
Lisa	2004
Bob	2008
Jake	2012
Sue	2014

Combining Datasets

Specifying Set Arithmetic for Joins

Inner join: default

```
df67_inner=pd.merge(df6, df7)
```

```
#df6
```

	name	food
0	Peter	fish
1	Paul	beans
2	Mary	bread

```
#df7
```

	name	drink
0	Mary	wine
1	Joseph	beer

```
#df67_inner
```

	name	food	drink
0	Mary	bread	wine

Combining Datasets

Specifying Set Arithmetic for Joins

```
df67_outer=pd.merge(df6, df7, how='outer')
```

```
#df6
```

```
name food
```

```
0 Peter fish
```

```
1 Paul beans
```

```
2 Mary bread
```

```
#df7
```

```
name drink
```

```
0 Mary wine
```

```
1 Joseph beer
```

```
# df67_outer
```

```
name food drink
```

```
0 Joseph NaN beer
```

```
1 Mary bread wine
```

```
2 Paul beans NaN
```

```
3 Peter fish NaN
```

Combining Datasets

Specifying Set Arithmetic for Joins

```
df67_left=pd.merge(df6, df7, how='left')
```

```
#df6
```

	name	food
0	Peter	fish
1	Paul	beans
2	Mary	bread

```
#df7
```

	name	drink
0	Mary	wine
1	Joseph	beer

```
# df67_left
```

	name	food	drink
0	Peter	fish	NaN
1	Paul	beans	NaN
2	Mary	bread	wine

Combining Datasets

Specifying Set Arithmetic for Joins

```
df67_left=pd.merge(df6, df7, how='right')
```

```
#df6
```

	name	food
0	Peter	fish
1	Paul	beans
2	Mary	bread

```
#df7
```

	name	drink
0	Mary	wine
1	Joseph	beer

```
# df67_right
```

	name	food	drink
0	Mary	bread	wine
1	Joseph	NaN	beer

Aggregation and Grouping

Planet data

Dataset: planet from seaborn

```
import seaborn as sns  
planets = sns.load_dataset('planets')  
print(planets.shape)  
print(planets.head())
```

(1035, 6)

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007

Simple Aggregation in Pandas

Get the overall properties of a dataset

```
print(planets.dropna().describe())
```

	number	orbital_period	mass	distance	year
count	498.00000	498.000000	498.000000	498.000000	498.000000
mean	1.73494	835.778671	2.509320	52.068213	2007.377510
std	1.17572	1469.128259	3.636274	46.596041	4.167284
min	1.00000	1.328300	0.003600	1.350000	1989.000000
25%	1.00000	38.272250	0.212500	24.497500	2005.000000
50%	1.00000	357.000000	1.245000	39.940000	2009.000000
75%	2.00000	999.600000	2.867500	59.332500	2011.000000
max	6.00000	17337.500000	25.000000	354.000000	2014.000000

Simple Aggregation in Pandas

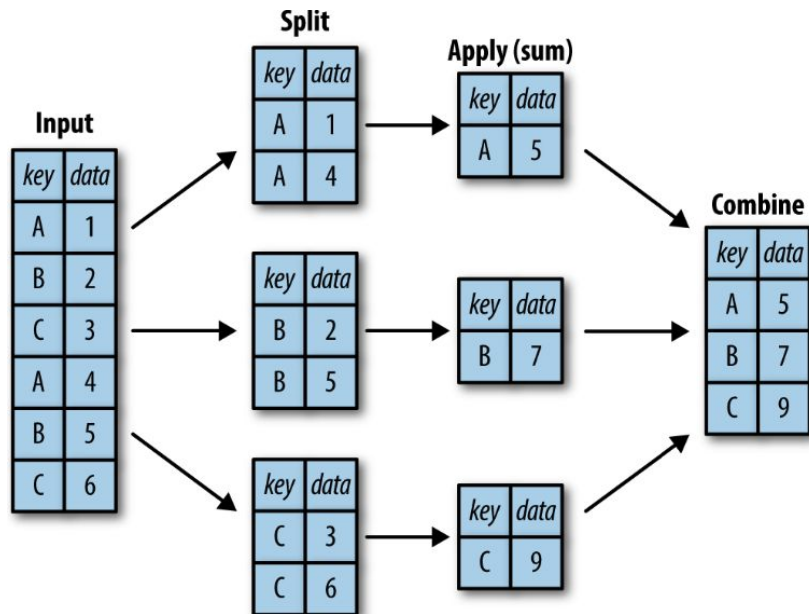
Table 3-3. Listing of Pandas aggregation methods

Aggregation	Description
<code>count()</code>	Total number of items
<code>first(), last()</code>	First and last item
<code>mean(), median()</code>	Mean and median
<code>min(), max()</code>	Minimum and maximum
<code>std(), var()</code>	Standard deviation and variance
<code>mad()</code>	Mean absolute deviation
<code>prod()</code>	Product of all items
<code>sum()</code>	Sum of all items

GroupBy: Split, Apply, Combine

A visual representation of a groupby operation: split, apply, combine

```
df = pd.DataFrame({  
    'key': ['A', 'B', 'C', 'A', 'B', 'C'],  
    'data': range(6)},  
    columns=['key', 'data'])  
df.groupby('key').sum()
```



GroupBy: Split, Apply, Combine

The GroupBy object: supports column indexing in the same way as the DataFrame, and returns a modified GroupBy object.

```
planets.groupby('method')['orbital_period'].median()
```

method	
Astrometry	631.180000
Eclipse Timing Variations	4343.500000
Imaging	27500.000000
Microlensing	3300.000000
Orbital Brightness Modulation	0.342887
Pulsar Timing	66.541900
Pulsation Timing Variations	1170.000000
Radial Velocity	360.200000
Transit	5.714932
Transit Timing Variations	57.011000

GroupBy: Split, Apply, Combine

The GroupBy object: supports column indexing in the same way as the DataFrame, and returns a modified GroupBy object.

```
planets.groupby('method')['year'].describe()
```

GroupBy: Split, Apply, Combine

Aggregate, filter, transform, apply

aggregate(): can take a string and compute all the aggregates at once.

```
print(df.groupby('key').aggregate(['min', 'median', 'max']))
```

	min	median	max	min	median	max
key						
A	0	1.5	3	3	4.0	5
B	1	2.5	4	0	3.5	7
C	2	3.5	5	3	6.0	9

#df	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

GroupBy: Split, Apply, Combine

Aggregate, filter, transform, apply

aggregate(): can take a string and compute all the aggregates at once.

```
print(df.groupby('key').aggregate({'data1': 'min', 'data2': 'max'}))
```

	data1	data2
key		
A	0	5
B	1	7
C	2	9

		data1	data2
#df	key		
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

GroupBy: Split, Apply, Combine

Aggregate, filter, transform, apply

Filtering: drop data based on filter condition

```
def filter_func(x):  
    return x['data2'].std() > 4
```

```
print(df.groupby('key').std())
```

```
print(df.groupby('key').filter(filter_func))
```

	data1	data2
key		
A	2.12132	1.414214
B	2.12132	4.949747
C	2.12132	4.242641

#df	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

	key	data1	data2
1	B	1	0
2	C	2	3
4	B	4	7
5	C	5	9

GroupBy: Split, Apply, Combine

Aggregate, filter, transform, apply

transform: return some transformed version of the full data, the output is the same shape as the input

```
print(df.groupby('key').transform(lambda x: x - x.mean()))
```

	data1	data2
0	-1.5	1.0
1	-1.5	-3.5
2	-1.5	-3.0
3	1.5	-1.0
4	1.5	3.5
5	1.5	3.0

	#df		
	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

pivot() and pivot_table()

pivot()

Data is often stored in so-called “stacked” or “record” format: one row for each subject.

Another representation:

the columns are the unique variables

Index identifies individual observations

another representation

variable A B C D

date

2020-01-03 0 3 6 9

2020-01-04 1 4 7 10

2020-01-05 2 5 8 11

stacked format

value variable date

0 0 A 2020-01-03

1 1 A 2020-01-04

2 2 A 2020-01-05

3 3 B 2020-01-03

4 4 B 2020-01-04

5 5 B 2020-01-05

6 6 C 2020-01-03

7 7 C 2020-01-04

8 8 C 2020-01-05

9 9 D 2020-01-03

10 10 D 2020-01-04

11 11 D 2020-01-05

pivot()

```
data = {  
    "value": range(12),  
    "variable": ["A"] * 3 + ["B"] * 3 + ["C"] * 3 + ["D"] * 3,  
    "date": pd.to_datetime(["2020-01-03", "2020-01-04", "2020-01-05"] * 4)  
}  
  
df = pd.DataFrame(data)  
  
pivoted = df.pivot(index="date", columns="variable", values="value")
```

pivot_table()

- `pivot()` can only handle unique rows specified by index and columns.
- If your data contains duplicates, use `pivot_table()`
- `pivot_table()`: pivot with aggregation of numeric data

pivot_table()

	A	B	C	D	E	F
0	one	aa	foo	-0.105748	-0.775077	2013-01-01
1	two	bb	foo	0.930743	1.181313	2013-02-01
2	three	aa	bar	0.422704	-1.049518	2013-03-01
3	one	bb	foo	-0.223289	-2.514550	2013-01-15
4	two	aa	foo	0.723952	-0.401466	2013-02-15
5	three	bb	bar	-0.037316	0.545797	2013-03-15

```
pd.pivot_table(df, values="D", index=["B"], columns=["C"])
```

C	bar	foo
B		
aa	0.422704	0.309102
bb	-0.037316	0.353727

pivot_table()

	A	B	C	D	E	F
0	one	aa	foo	-0.105748	-0.775077	2013-01-01
1	two	bb	foo	0.930743	1.181313	2013-02-01
2	three	aa	bar	0.422704	-1.049518	2013-03-01
3	one	bb	foo	-0.223289	-2.514550	2013-01-15
4	two	aa	foo	0.723952	-0.401466	2013-02-15
5	three	bb	bar	-0.037316	0.545797	2013-03-15

```
pd.pivot_table(df, values=["D", "E"], index=["B"],  
               columns=["A", "C"], aggfunc="sum",)
```

	D		E			
A	one	three	two	one	three	two
C	foo	bar	foo	foo	bar	foo
B						
aa	-0.105748	0.422704	0.723952	-0.775077	-1.049518	-0.401466
bb	-0.223289	-0.037316	0.930743	-2.514550	0.545797	1.181313